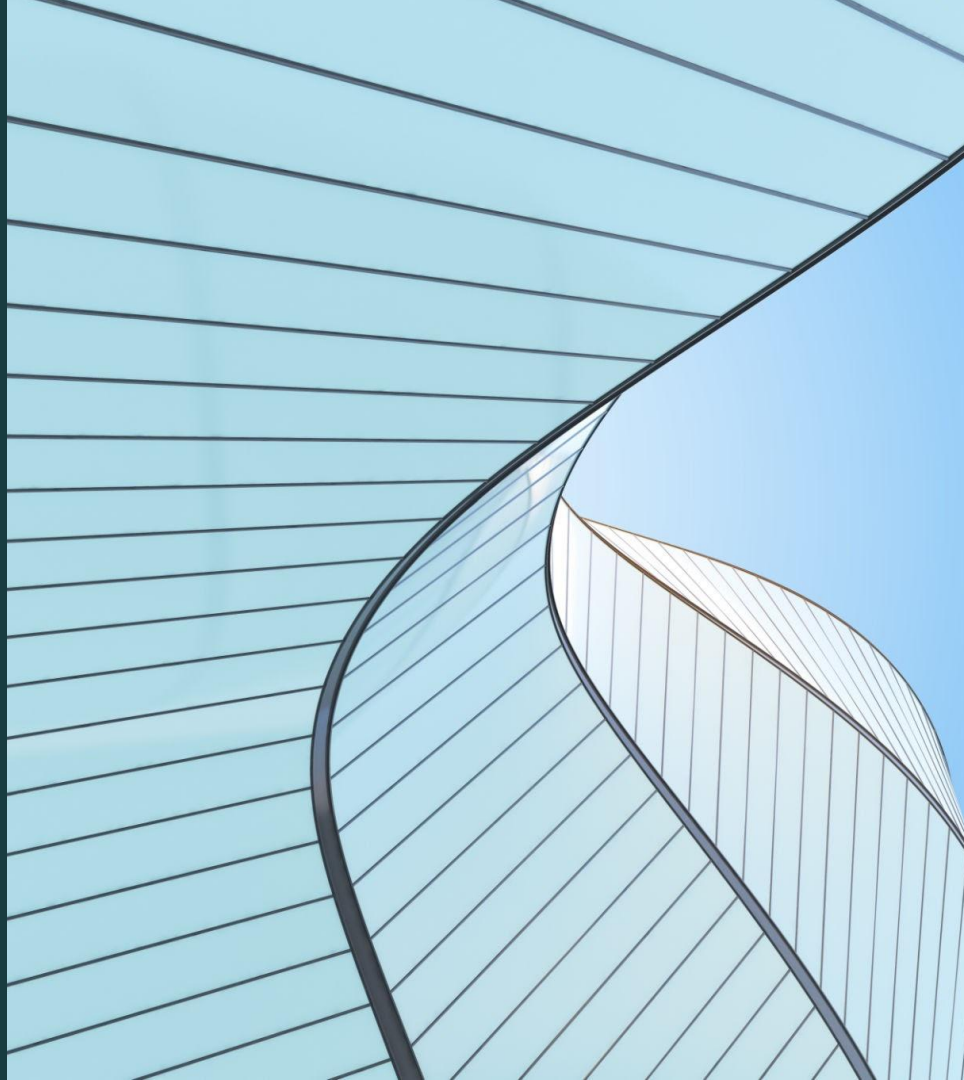
Mahmood Saadat
Jan 2026

Embedded C
A Modular Approach

Embedded C, Elevated:
Build robust, maintainable firmware with structured modules, coding standards, and clear software layers

https://cosmorobotics.uk



Course Objectives
Why modular design?

1

Transitioning from "Code that just works" to "Code that scales."

2

Mastering the 4-layer architecture: HAL, Device, Interface, and Application.

3

Adopting professional industry standards (Barr Group C).

4

Developing a reusable project template you can take to any job.

The "Electronics Engineer" Coding Style

The Hardware Mental Trap: The Monolithic Approach

The Symptom:

Everything lives in main.c or a single giant interrupt.

The Problem:

Hard-coding platform specific values directly into application logic.

The Result:

"Spaghetti Code" that is impossible to debug and even harder to port to a new MCU.

The Cost:

If the chip changes, you have to rewrite the entire project from scratch.

Shifting Your Perspective

Hardware is Temporary, Logic is Permanent

Decoupling	→	Separating What the system does from How the hardware executes it.
Mindset	→	Thinking in "Drivers" and "Modules" instead of "Pins" and "Bits."
The goal	→	Your Application Layer should not know if it's running on an STM32, an ESP32, or a PIC.

The Roadmap to Professional Firmware

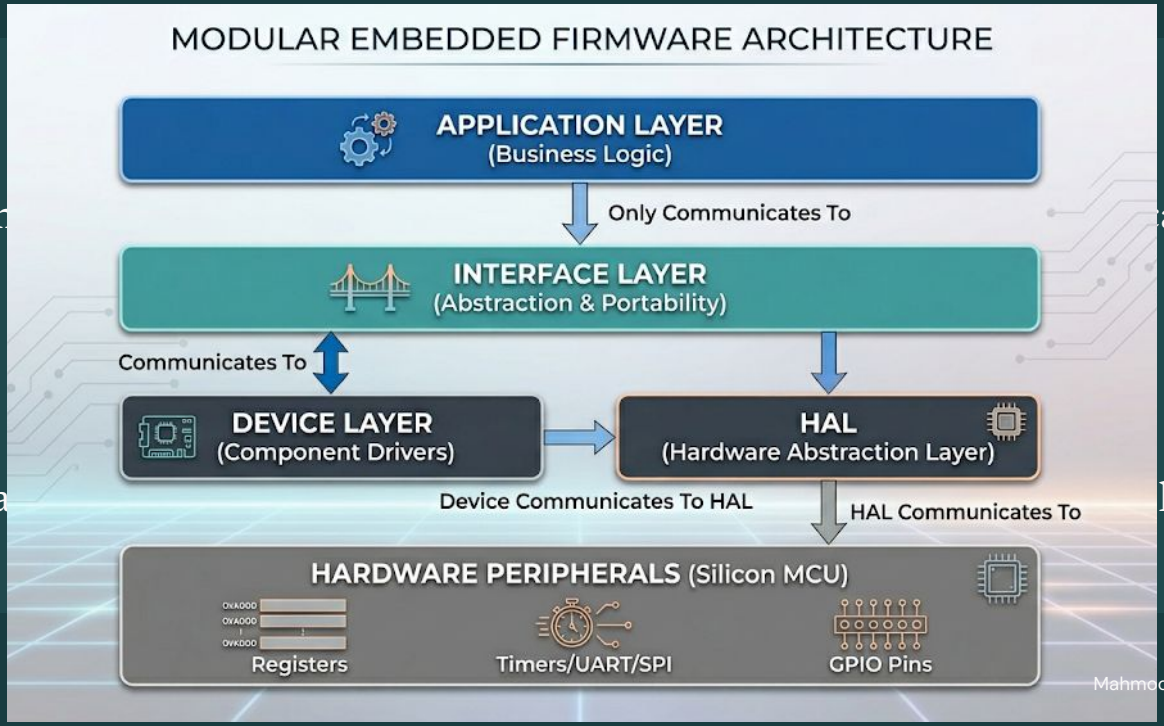
Our Core Framework: The 4-Layer Stack

1

HAL: The silicon … al components

3

Interface: The la… logic and portability.



Why Standards Matter?
Code Quality is Hardware Safety

1

Software bugs in embedded systems can lead to physical damage.

2

Reducing "Mental Load" so you can focus on logic, not syntax.

3

A coding standard is like a PCB Design Rule Check (DRC). It ensures the final product is manufacturable and reliable.

4

Which standard to pick?

Rules for Variable Types

Eliminating Ambiguity with Fixed-Width Types

The Problem:

int can be 16-bit or 32-bit depending on the compiler.

The Solution:

Use <stdint.h> for all variable declarations.

The Rule:

Always use uint8_t, int16_t, and uint32_t instead of char, short, or long.

Why it matters:

Prevents overflow errors when porting code between different microcontrollers.

Naming Conventions & Readability

Making Code "Self-Documenting"

Consistency:

Adopting a clear naming convention for variables, functions, and constants.

The Rule:

Descriptive names over short ones (e.g., motor_stop() vs. m_stp()).

Brace Placement:

Consistent use of braces even for single-line if statements to prevent logic errors.

Commentary:

Documenting the Why behind the code, not the What (the code should tell us what it's doing).

Safe Pointer & Memory Usage

Preventing System Crashes

Rule 1:

Initialize all variables and pointers immediately.

Rule 2:

Always check for NULL before dereferencing a pointer.

Avoiding "Magic Numbers":

Use #define or const for hardware addresses and bitmasks.

Keyword volatile:

When and why to use it for peripheral registers and global flags shared with Interrupt Service Routines (ISRs).

Rules

What you will see in my code

- Add prefix to identify the variable

- g_ for global variables	- b_ for boolean
- p_ for pointer	- a_ for array
- t_ for function arguments	

So, if you see g_p_a_b_is_valid, this will show that variable is global, it's a pointer array of type boolean. I don't include any other types in the variable name.

Comparing codes

```
unsigned int

void main() {
    TRISA = 0
    while(1)
        if(PO
            r
    }
}
```

```c
#include <stdint.h>
#include "hal_gpio.h"

#define SENSOR_ACTIVE 1
#define INPUT_PIN_0   0

uint8_t g_sensor_status = 0; // Global starts with g_

void main(void)
{ // Braces on their own line
    hal_gpio_init(); // Lower snake case

    while (1)
    {
        g_sensor_status = hal_gpio_read_pin(PORT_A, INPUT_PIN_0);

        // Constant on the left (Yoda Condition)
        if (SENSOR_ACTIVE == g_sensor_status)
        {
            motor_start();
        }
    }
}
```

The Philosophy of Layering
The Power of "Separation of Concerns"

Portability:

Swapping an STM32 for an ESP32 only affects the HAL layer.

Safety:

The Application never touches a register; it can’t accidentally "brick" the hardware.

Readability:

Code looks like a story, not a datasheet.

Unit Testing:

You can test the Application logic on your PC by "mocking" the Interface layer. Or even better, replace the HAL and mimic the hardware!

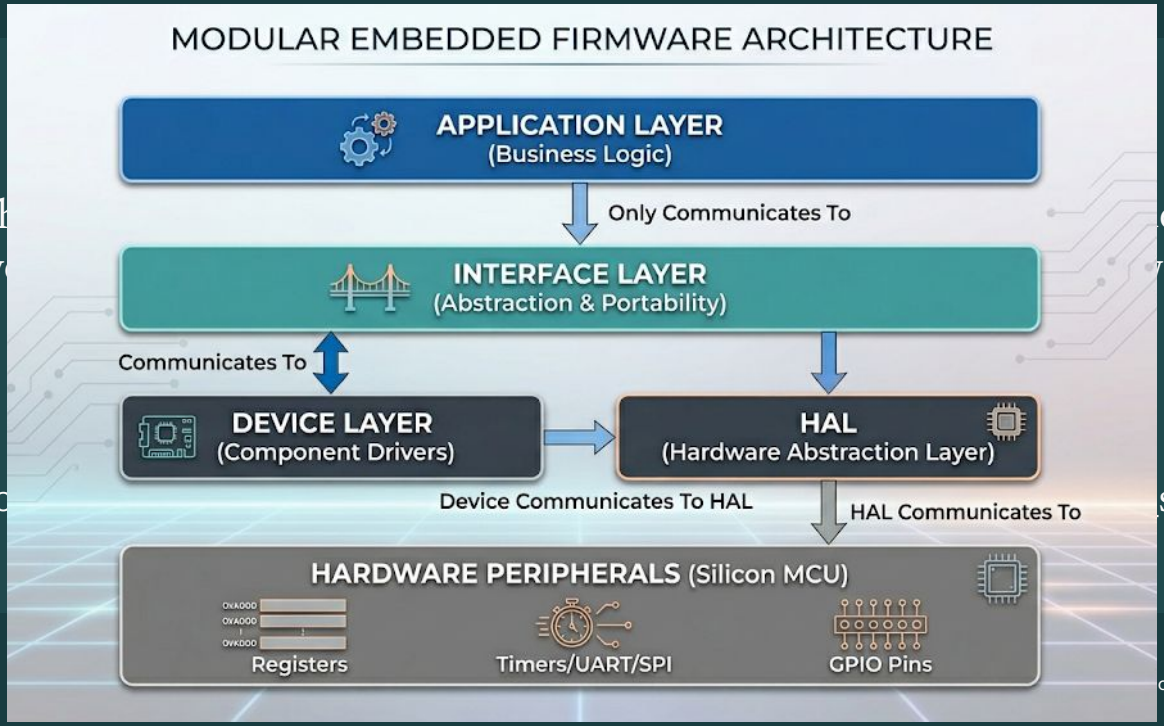
The Application Layer (The Brain)
Designing the "Hardware-Agnostic" Brain

Content:

Contains the high ency like empty, open valv ved here.

Interaction:

Only calls functic status) Layer.



The Interface Layer (The Translator)

The Interface: Your Universal API

Standardizes how the Application talks to the outside world.

Provides a generic "Contract" (e.g., sensor_get_data()).

It maps generic commands to the specific Device or HAL functions using parameters.

The Device Layer (Component Drivers)
Modeling the Physical Components

The Goal:

Create a "Software Object" for every part on your PCB.

The Rule:

This layer handles component-specific logic (e.g., initialisation sequences or data conversion formulas).

Communication:

The Device layer calls the HAL/Interface to use the peripherals, but it doesn't care which MCU is doing the work.

Examples:

eeprom_m24cxx, sensor_bme280, or oled_ssd1306.

The HAL (Hardware Abstraction Layer)
Modeling the Physical Peripherals

The Goal:

Create a "Software Object" for every peripheral on your MCU.

The Rule:

This is the only layer allowed to include MCU-specific headers (e.g., stm32f4xx.h) in its source files (not the headers).

Responsibility:

Managing internal peripherals like GPIO, UART, I2C, and Timers.

Examples:

void hal_gpio_write(hal_gpio_t * t_p_handle, bool t_state);

The Responsibility of the HAL
The HAL: Your MCU's "Service Manual"

Goal:

To provide a standard way to access hardware without knowing the register addresses.

Scope:

Covers internal peripherals only (GPIO, UART, SPI, I2C, Timers, ADC).

Barrier:

This is the only place in the entire project where you should see a register name or any access to a MCU related libraries.

Benefit:

If the chip changes, the Application and Device layers remain 100% untouched.

Designing a HAL
Creating the Hardware Contract

Header file:

Create a template for HAL library. In these templates, the header file will stay the same across different MCUs.

Source:

This is the only place we can put anything platform/MCU specific.

Data Types:

Always define typedef for the configuration of the HAL and any enum needed in this library.

Maintenance:

Use a separate repository for your HAL. Include this repository as a sub-module in your main project.

What is a "Device" in Firmware?
Moving from Pins to Products

The Definition:

A Device is any physical component on your PCB that requires logic to operate.

The Goal:

To create a "Driver" that describes how that component behaves, regardless of which MCU it is plugged into.

The Rule:

The Device Layer calls the HAL to perform the physical action (like toggling a pin).

Examples:

An LED, a 16x2 LCD, an EEPROM, or a Motor Driver.

Designing a Device library
Creating the Object Contract

Header file:

Use a template for Devie library. In these templates, the header file describes the functionality supported by device.

Source:

Add the code to perform the functionality needed for the device.

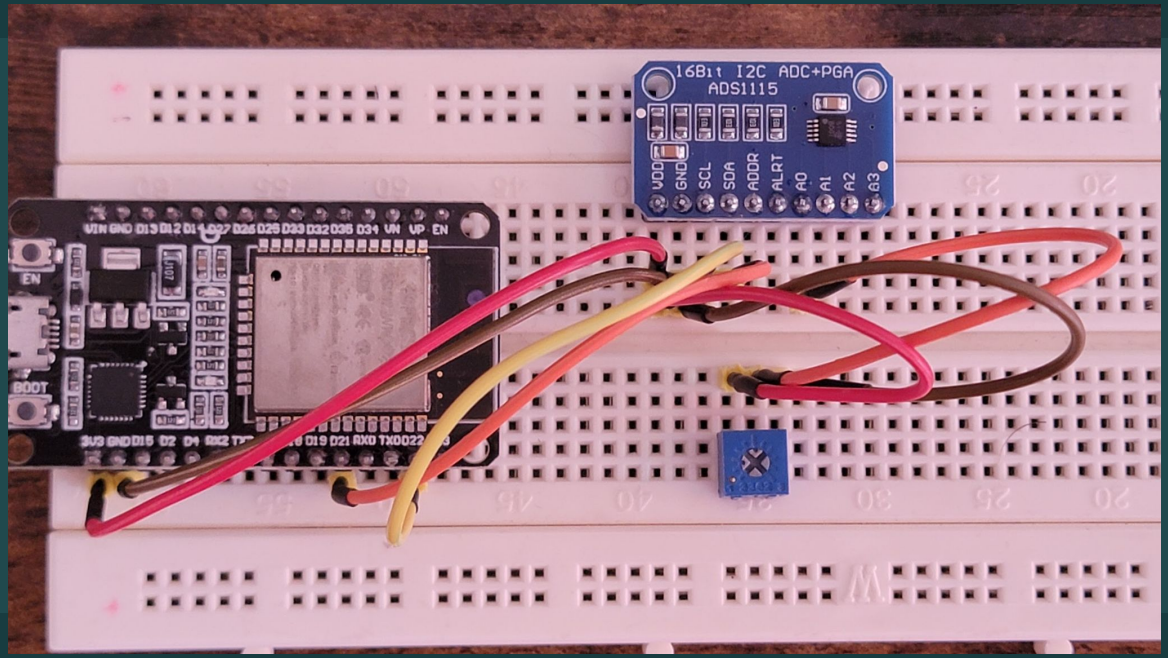
Data Types:

Always define typedef for the configuration of the Device and any enum needed in this library.

Maintenance:

Use a separate repository for the Devices. Include this repository as a sub-module in your main project.

Example
Connecting to ADS115



Why use the Interface Layer?
The "Universal API" of Your Firmware

The Goal:

To provide a generic API that never changes, even if the hardware does.

The Concept:

The Application asks to power_on(); it doesn't care if it's a Relay, a MOSFET, or a Chip-Enable pin.

The Rule:

This layer "hides" the Device and HAL layers from the Application.

The Benefit:

You can write your entire Application logic before the PCB is even manufactured!

The Brain of the System
The Application: Where Logic Lives

The Goal:

Focus entirely on the product's features and user requirements.

The Rule:

This layer is 100% portable C code. It should compile on a PC just as easily as an MCU.

Communication:

It only talks to the Interface Layer.

States:

It manages the "State" of the machine (e.g., Idle, Processing, Error).

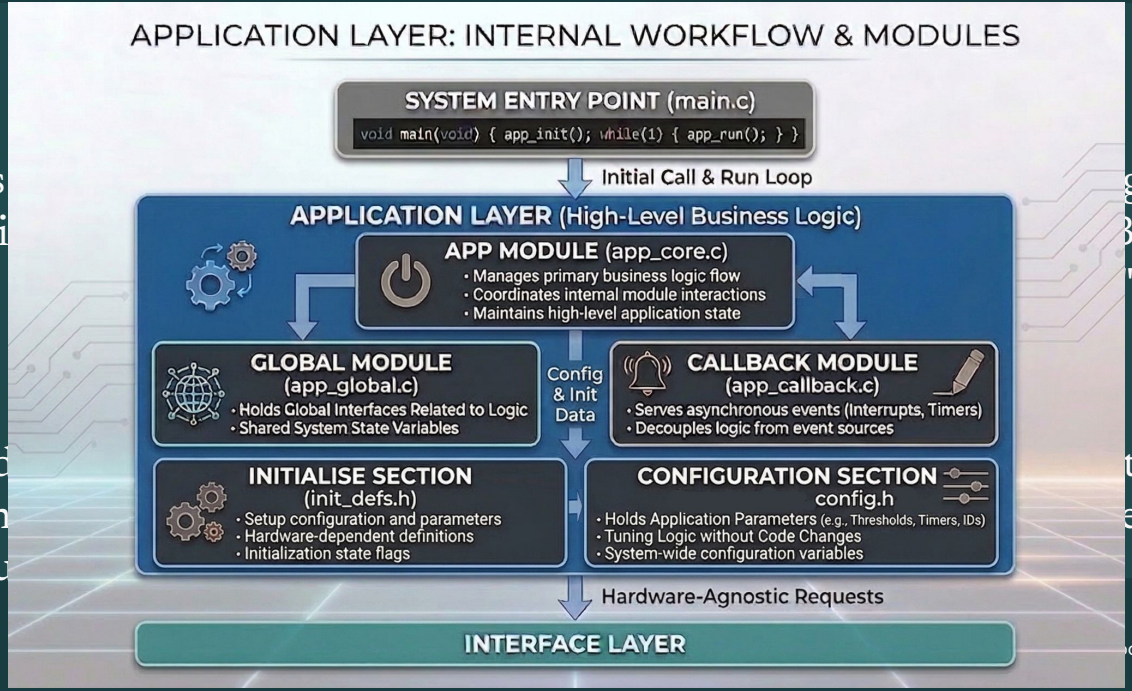
Anatomy of the Application Layer
Inside the Application Layer: Structure and Flow

The Orchestrator:

The main.c file has
Application initiali
gers the hardware
Business Logic"
").

The Callback Module:

A reactive system d
asynchronous even
user inputs) withou
the global
e project's



Organizing Your Workspace
Structuring the File System

The Goal:

A clean folder structure that matches the 4-layer architecture.

The Rule:

Keep Header and Source files separated or grouped by layer.

The Folder Tree:

- **/app:** High-level logic and state machines.
- **/interface:** The abstraction "wrappers."
- **/device:** Drivers for external hardware (LCD, Sensors).
- **/hal:** Silicon-specific drivers and register maps.
- **/common:** Shared types and functions

Version Control & Portability
Future-Proofing Your Firmware

The Rule:

If you change the MCU, you only replace the files in the /hal folder.

The Benefit:

90% of your code remains untouched and tested.

Layers:

Using git to track changes in specific layers.

Template:

Creating a "Base Project Template" that you can copy-paste for every new job.

Documenting for the Next Engineer
Self-Documenting Code & Doxygen

The Goal:

Using standardized comment blocks for every function.

The Rule:

Every function must state its:

- @brief: What does it do?
- @param: What are the inputs?
- @return: What is the hal_status_t result?

Using AI
What to consider?

The Goal:

Making development easier

The Rule:

- Define your standard
- Define template
- Define the architecture
- Let the AI only handling the typing part of the code

From Electronics Engineer to Firmware Architect

- You have moved beyond "Spaghetti Code."
- You now have a reusable, 4-layer framework.
- You follow the Barr Group safety standards.
- You are ready to build professional, scalable, and portable embedded systems.